Лабораторная работа №7. Коллективные операции в MPI

Марданов Самир, группа 11-401

Постановка и реализация. Вычисляется сумма массива $a_i = i + 1$, $i = 0, \ldots, N - 1$. Процесс 0 читает N из аргумента командной строки и передаёт его всем процессам через `MPI_Bcast`. При $N > 0$ и $N \bmod P = 0$ корневой процесс формирует массив, а `MPI_Scatter` распределяет по N/P последовательных элементов каждому процессу. Каждый процесс суммирует свою часть; суммы имеют тип `long long`, MPI-тип — `MPI_LONG_LONG_INT`.

`MPI_Gather` собирает локальные суммы на процессе 0, где они складываются циклом; `MPI_Reduce` с `MPI_SUM` возвращает итог только процессу 0; `MPI_Allreduce` с той же операцией возвращает итог всем процессам. Процесс 0 сравнивает три результата с $S = N(N + 1)/2$ и выводит их. Статус проверки передаётся всем процессам через `MPI_Bcast`. Все коллективные вызовы выполняются в одинаковом порядке. При ошибочных аргументах или нехватке памяти процессы согласованно завершаются через `MPI_Finalize`.

Компиляция и запуск. macOS arm64, MPICH 5.0.2, компилятор Clang через `mpicc`; все процессы на одном компьютере.

```
mpicc -O2 -Wall -Wextra -std=c11 mpi_array_sum.c -o lab7
mpiexec -n 4 ./lab7 1000000
```

3.1. Проверка корректности

P	N	Ожидается	Gather	Reduce	Allreduce	Верно?
2	100	5050	5050	5050	5050	да
4	100	5050	5050	5050	5050	да
4	10000	50005000	50005000	50005000	50005000	да
8	1000000	500000500000	500000500000	500000500000	500000500000	да

Также проверены $N = 1$, $P = 1$ и завершение без зависания для отсутствующего аргумента, $N = 0$, $N = -1$, $N = 101$ при $P = 4$, нечислового ввода, переполнения `int` и лишних аргументов.

Методика измерений. В таблицах — среднее пяти отдельных запусков каждой конфигурации; все результаты корректны. После начального барьера `MPI_Wtime` измеряет выделение памяти, заполнение массива, распределение, локальное суммирование и три способа агрегации; берётся максимальное время процессов. Инициализация MPI, передача N, вывод и завершение вне замера. $S_P = T_1/T_P$, $E_P = S_P/P$.

3.2. Влияние N, $P = 4$

N	Время, с	Верно?
10000	0.0002332	да
100000	0.0003124	да
1000000	0.0010882	да
10000000	0.0102494	да

3.3. Влияние P, $N = 10^6$

P	Время, с	S_P	E_P
1	0.0007914	1.000	1.000
2	0.0009224	0.858	0.429
4	0.0010882	0.727	0.182
8	0.0033566	0.236	0.029

3.4. Сравнение Gather и Reduce

Оба способа дают ожидаемую сумму. Для Gather нужен массив из P локальных сумм на корне и дополнительный цикл сложения $O(P)$; Reduce выражает суммирование одним коллективным вызовом и не требует этого массива. Отдельное время агрегации не измерялось; сравнительная скорость этих операций по приведённым замерам не устанавливается.

Вывод. Gather собирает отдельные значения, Reduce объединяет их на корне, Allreduce возвращает объединённый результат всем. Allreduce удобен, когда дальнейшим вычислениям каждого процесса нужна общая сумма: отдельная рассылка результата не требуется. В выполненных экспериментах увеличение P не дало ускорения: распределение данных, коллективные операции и последовательная подготовка массива на корне оказываются существеннее выигрыша локального суммирования. При росте N время растёт; выводы относятся к данной среде и измеряемому участку программы.